

Column-Oriented Storage — Revision Notes

Source: *Designing Data-Intensive Applications* (Martin Kleppmann), Chapter 3, "Transaction Processing or Analytics?" and "Column-Oriented Storage" (pp. 90–103). Related: [SSTables-and-LSM-Trees.md](#) · [B-Tree.md](#)

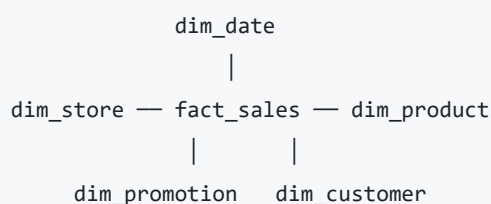
1. Why we need it: OLTP vs OLAP

Databases serve two very different kinds of work:

	OLTP (transaction processing)	OLAP (analytics)
Who uses it	End users, through the application	Analysts, for business decisions
Typical query	Fetch a few rows by key (<code>WHERE id = 42</code>)	Aggregate over millions of rows (<code>SUM</code> , <code>COUNT</code> , <code>AVG</code>)
Writes	Random, low-latency, small	Bulk loads (ETL) or event streams
Data	Current state	History of events over time
Size	GB to TB	TB to PB
Bottleneck	Disk seek time	Disk bandwidth (reading lots of data)
Storage layout	Row-oriented (B-trees, LSM-trees)	Column-oriented

- Analytics runs in a separate **data warehouse**, so heavy queries don't slow down the live OLTP system.
- Data gets there by **ETL** (Extract → Transform → Load): copy from OLTP databases, clean it up, and load it into the warehouse.
- Examples: Teradata, Vertica, SAP HANA, Amazon Redshift, and SQL-on-Hadoop engines like Hive, Spark SQL, Impala, Presto and Drill.

2. The warehouse data model: star and snowflake schemas



- **Fact table** (`fact_sales`): one row per **event** (e.g. one purchase). It can hold **trillions of rows / petabytes**.
- **Dimension tables** (`dim_product` , `dim_store` , `dim_date` ...) describe the **who, what, where, when, how and why** of each event. Fact rows point to them with foreign keys.
- **Star schema**: the fact table sits in the middle with dimensions around it, like rays of a star.
- **Snowflake schema**: dimensions are split further into sub-dimensions (e.g. `dim_product` → `dim_brand` , `dim_category`). It's more normalized, but analysts usually prefer the simpler star.
- Tables are **very wide**: fact tables often have **100+ columns**, sometimes several hundred.

3. The problem with row-oriented storage

A typical analytics query reads **many rows but only 4–5 columns**:

```
SELECT dim_date.weekday, dim_product.category,  
       SUM(fact_sales.quantity) AS quantity_sold  
FROM fact_sales  
  JOIN dim_date   ON fact_sales.date_key   = dim_date.date_key  
  JOIN dim_product ON fact_sales.product_sk = dim_product.product_sk  
WHERE dim_date.year = 2013  
      AND dim_product.category IN ('Fresh fruit', 'Candy')  
GROUP BY dim_date.weekday, dim_product.category;
```

From `fact_sales`, it needs only `date_key`, `product_sk` and `quantity`.

- In a **row-oriented** store, all values of one row sit **next to each other** on disk.
- Even with indexes, the engine must **load whole rows (100+ columns)** from disk, parse them, and throw most of it away. That's very slow.

4. The idea: store each column separately

Don't store all the values of a row together. Store all the values of each column together, usually one file per column.

Row-oriented (one row after another):

```
[date=140102, product=69, store=4, qty=1, price=13.99] [date=140102, product=69, store=5, qty=3, ...]  
...
```

Column-oriented (one file per column):

date_key	file:	140102,	140102,	140102,	140102,	140103,	140103,	...
product_sk	file:	69,	69,	69,	74,	31,	31,	...
store_sk	file:	4,	5,	5,	3,	2,	3,	...
quantity	file:	1,	3,	1,	5,	1,	3,	...

- A query reads **only the column files it needs**, which saves a huge amount of disk I/O.
- **Every column file stores the rows in the same order.** To rebuild row 23, take the **23rd value from each column file**.
- It's easiest to picture with relational tables, but it works for other data too. **Parquet** is a columnar format that supports a **document data model**, based on Google's **Dremel**.

5. Column compression

Column values are often **very repetitive** (the same product, store or date over and over), so they **compress very well**.

Bitmap encoding

A column usually has **few distinct values** compared to the number of rows (billions of sales, but maybe 100,000 products).

- For a column with **n distinct values**, make **n bitmaps**: one per value, with **one bit per row** (1 = this row has that value).

```
product_sk column: 69 69 69 69 74 31 31 31 31 29 30 30 31 31 31 68 69 69
```

```
value 29: 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 → run-length: 9,1
value 30: 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 → run-length: 10,2
value 31: 0 0 0 0 0 1 1 1 1 0 0 0 1 1 1 0 0 0 → run-length: 5,4,3,3
value 68: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 → run-length: 15,1
value 69: 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 → run-length: 0,4,12,2
value 74: 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 → run-length: 4,1
```

- Small n** (e.g. ~200 countries): store the bitmaps as plain bits.
- Large n**: the bitmaps are mostly zeros (**sparse**), so also apply **run-length encoding** (RLE). "9,1" means 9 zeros, then 1 one, then zeros for the rest.

Bitmaps make filters fast

Query	How it's answered
WHERE product_sk IN (30, 68, 69)	Load the 3 bitmaps and take their bitwise OR
WHERE product_sk = 31 AND store_sk = 3	Load both bitmaps and take their bitwise AND . This works because the k-th bit is the same row in every column

⚠ Column families are NOT column-oriented

Cassandra and HBase have *column families* (from Bigtable), but inside each family they store **all columns of a row together** with the row key, and they **don't compress columns**. So the Bigtable model is **still mostly row-oriented**.

6. Memory bandwidth and vectorized processing

Disk → memory is not the only bottleneck. Analytical databases also care about **memory** → **CPU cache** bandwidth, branch mispredictions, CPU pipeline stalls, and using **SIMD** instructions.

- The engine takes a **chunk of compressed column data** that fits in the CPU's **L1 cache** and loops over it **tightly**, with no function calls per row.
- Compression lets **more rows fit** in the same cache.
- Operators like bitwise AND/OR can work **directly on compressed chunks**.

This is called **vectorized processing**.

7. Sort order in column storage

- By default, rows are kept in **insertion order**, so a new row is just appended to every column file.
- You can instead **sort** them (like SSTables) and use the order as an **index**.
- ⚠️ You **can't sort each column independently**, or you'd lose track of which values belong to the same row. **Sort entire rows**, even though they're stored by column.

Choosing sort keys:

- **First sort key** = what queries filter on most, e.g. `date_key` for "last month" queries, so the engine scans only that date range.
- **Second sort key** breaks ties, e.g. `product_sk`, so sales of the same product on the same day sit together.

Bonus: sorting improves compression.

- After sorting, the first sort column has **long runs of the same value**. RLE can shrink it to **a few KB, even for billions of rows**.
- The effect is strongest on the **first** sort key. Later sort keys are more jumbled and compress less, but it's still a win overall.

Several different sort orders (C-Store / Vertica)

- Data is **replicated** to several machines anyway for fault tolerance, so **store each replica sorted differently**.
- Each query uses the replica whose sort order fits it best.
- This is like having several secondary indexes in a row store. The difference is that a row store keeps each row in one place and indexes hold **pointers**, while a column store has **no pointers**, only columns of values.

8. Writing to column-oriented storage

Compression and sorting make reads fast but **writes hard**:

- **Update-in-place (like B-trees) is impossible** with compressed columns.
- Inserting a row in the middle of a sorted table would mean **rewriting every column file**, because rows are identified by their position.

Solution: use LSM-trees.

```
writes → in-memory store (sorted; row- or column-oriented)
      |
      | when enough writes accumulate
      ▼
      merge with column files on disk → write NEW column files in bulk
reads  → combine column files on disk + recent writes in memory
```

- This is essentially what **Vertica** does.
- The query optimizer hides the two parts, so analysts see inserts, updates and deletes **immediately**.

9. Aggregation: materialized views and data cubes

Warehouse queries use aggregates (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`) a lot. Why recompute them from raw data every time? **Cache them.**

	Virtual view	Materialized view
What it is	A saved query (a shortcut)	An actual copy of the results on disk
On read	Expands into the underlying query and runs it	Reads the precomputed results
On write	Nothing extra	Must be updated when the data changes, so writes cost more
Good for	Anywhere	Read-heavy warehouses (rare in OLTP)

Data cube (OLAP cube)

A grid of aggregates grouped by several dimensions (example numbers):

	product 31	product 32	product 33	total by date
date 140101	149.60	31.01	84.58	265.19
date 140102	132.18	19.98	82.91	235.07
date 140103	196.75	48.50	69.99	315.24
total by product	478.53	99.49	237.48	815.50

- Each cell = e.g. `SUM(net_price)` for that date-product combination. Summing a row or column removes one dimension.
- Real facts have more dimensions (date, product, store, promotion, customer), giving a 5-dimensional **hypercube**, but the idea is the same.
- ☒ **Pro:** some queries become instant, e.g. "total sales per store yesterday" is just a lookup.
- ☒ **Con: less flexible.** You can't ask "what share of sales came from items over \$100?" because price isn't a dimension.
- So warehouses keep **as much raw data as possible** and use cubes only as a **performance boost** for common queries.

10. Where you'll see it in practice

Kind	Examples
Columnar warehouses	Amazon Redshift, Vertica, Google BigQuery*, Snowflake*, ClickHouse*
Columnar file formats	Apache Parquet (Dremel-based), Apache ORC*
Query engines on columnar files	Hive, Spark SQL, Impala, Presto, Drill

* Not in the book, but common in interviews.

11. Conclusion

Analytics queries scan **millions of rows but only a few columns**, so **column-oriented storage** keeps each column in its own file, with every file in the **same row order**. Queries read only the columns they need. Repetitive column values **compress very well**, especially with **bitmap encoding + run-length encoding**, and bitmaps make `IN` / `AND` filters simple bitwise operations. Compressed chunks fit in the CPU cache for fast **vectorized processing**. Rows can be **sorted** (whole rows, not single columns) to speed up range filters and improve compression, and C-Store/Vertica even keeps **different sort orders on different replicas**. Writes are hard (no update-in-place), so column stores buffer writes in memory and merge them into new files, the **LSM-tree** approach. **Materialized views and data cubes** precompute common aggregates, trading flexibility and write cost for speed. Remember that **Cassandra/HBase column families are still row-oriented**.

Interview one-liners

- **Column storage:** "Store values column by column, so analytic queries read only the columns they touch. Repetitive columns compress well, which makes OLAP scans fast."
- **Row vs column:** "Row stores suit OLTP: fetch or update a whole record by key. Column stores suit OLAP: aggregate a few columns over huge numbers of rows."
- **Writes:** "Column stores can't update in place, so they buffer writes in memory and merge them into new column files, like an LSM-tree."

Self-check questions

1. How do OLTP and OLAP workloads differ? Why use a separate data warehouse?
2. What are fact and dimension tables? Star vs snowflake schema?
3. Why is a row-oriented store slow for analytics queries?
4. How does a column store rebuild a full row?
5. Explain bitmap encoding and run-length encoding with an example.
6. How do bitmaps answer `IN (...)` and `AND` filters?
7. Why are Cassandra/HBase column families not really column-oriented?
8. What is vectorized processing?
9. Why can't you sort each column independently? How does sorting help compression?
10. Why does C-Store/Vertica store different sort orders on different replicas?
11. Why are writes hard in a column store, and how do LSM-trees solve it?
12. Materialized view vs virtual view? What are the pros and cons of a data cube?